

RESOLVEDESK PLATFORM

End-to-End System Architecture, API Specifications, State Machine & Database Blueprint

Project Name:	ResolveDesk Customer Support & Ticketing System
Backend Tech:	Java Spring Boot REST API & Supabase PostgREST Engine
Frontend Tech:	Vanilla JS, HTML5, Modern CSS Design Tokens
Database System:	Supabase PostgreSQL 15+ (Relational Schema)
Prepared By:	Viraj (Lead Author & Systems Architect)
Academic Year / Dept:	Year III — Dept of Information Technology
Institution:	J.J College of Engineering and Technology
Date of Release:	September 2026

EXECUTIVE SUMMARY:
ResolveDesk eliminates ticket ownership ambiguity, enforces server-validated ticket state transitions, manages priority SLA windows, and maintains real-time conversation threads between customers and support agents.

1. System Overview & Architecture

ResolveDesk is designed as a decoupled multi-tier enterprise architecture. The frontend client connects via HTTPS to a lightweight reverse proxy engine (Express/Node.js), which routes REST API calls to a high-performance Java Spring Boot backend service. The backend implements strict Role-Based Access Control (RBAC) using JSON Web Tokens (JWT) and communicates with a Supabase PostgreSQL database using RestTemplate PostgREST integration.

Architectural Layer	Technology Stack	Core Responsibilities
Presentation Client	HTML5, Vanilla JS, CSS Glassmorphism	Role-specific portals (Customer Dashboard, Agent Queue, Admin Panel), interactive ticket threads, OTP reset UI.
API Gateway / Proxy	Express.js / Node.js Proxy	Exposes port 3000, proxies REST calls to Spring Boot, serves client static web assets.
Backend Services	Java Spring Boot (Port 8080)	JWT verification, role authorization, state transition state machine validation, business logic.
Database Layer	Supabase PostgreSQL 15+	Relational storage, foreign key constraints, sequence identity PKs, triggers & status history logging.

2. Ticket State Machine Workflow

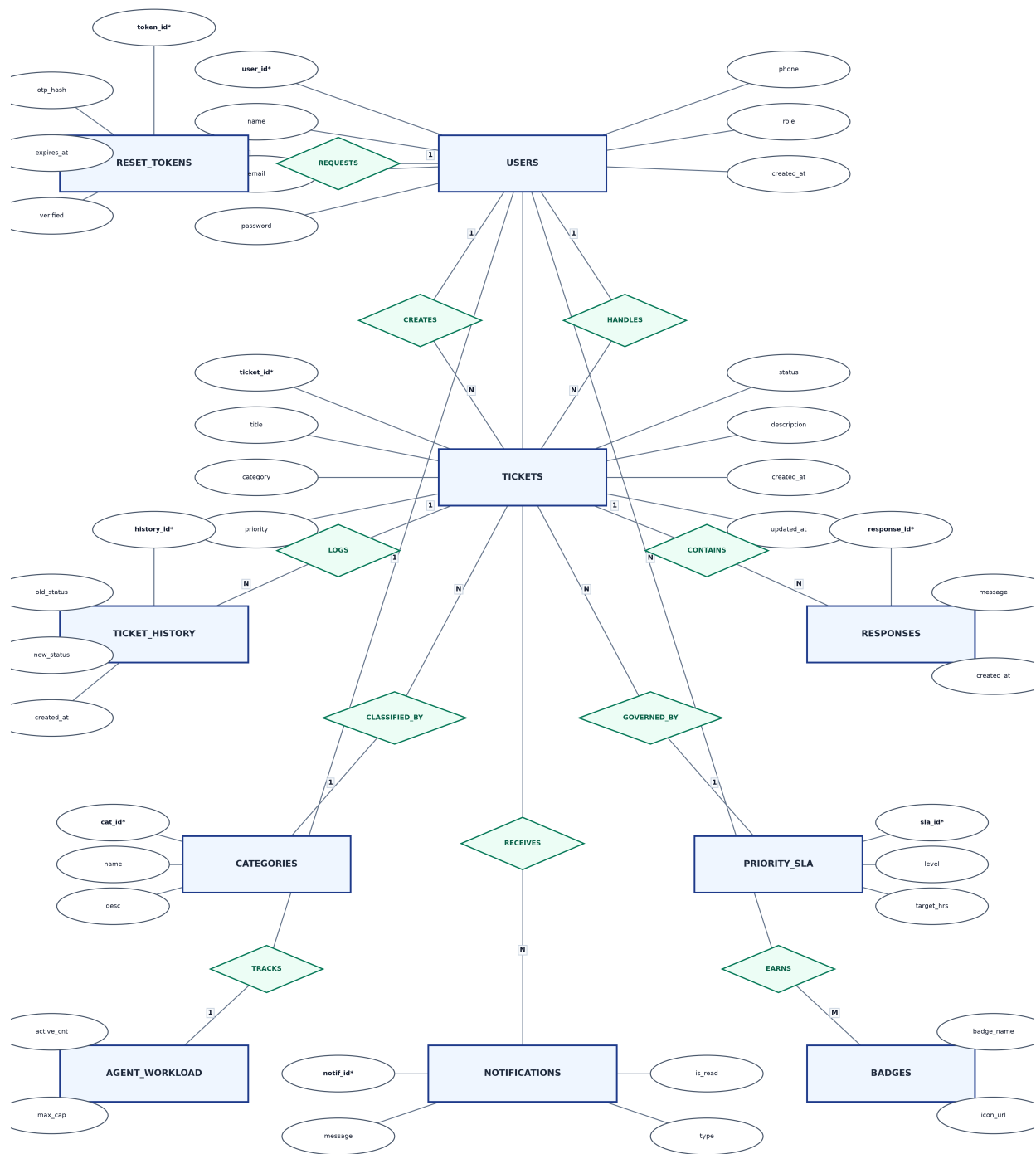
To prevent irregular support operations, ResolveDesk enforces a legal state machine on the backend. Attempts to skip state transitions or modify tickets without assigned owners are rejected with HTTP 400/403.

- **OPEN:** Ticket created by Customer. Unassigned.
- **ASSIGNED:** Admin assigns ticket to an Agent. Agent assigned_agent_id is bound.
- **IN_PROGRESS:** Agent commences work or posts response thread updates.
- **RESOLVED:** Agent completes fix and submits resolution for Customer review.
- **CLOSED:** Customer accepts fix, closing the issue thread permanently.
- **REOPENED:** Customer rejects resolution (requires non-empty feedback reason comment); ticket returns to IN_PROGRESS.

3. Traditional ER Diagram & Conceptual Schema

The conceptual Entity-Relationship (ER) model below illustrates all 12 core system entities, their primary/foreign key attributes, relationship types, and cardinalities.

TRADITIONAL ER DIAGRAM
Customer Support and Ticketing System (ResolveDesk)



4. REST API Endpoint Reference

Method	Endpoint Path	Role Allowed	Description & Operations
POST	/auth/signup	Public	Registers new customer or agent user account.
POST	/auth/login	Public	Authenticates credentials and returns signed JWT token.
POST	/tickets	Customer	Creates a new support ticket with category & priority.
GET	/tickets/my	Customer	Fetches all tickets created by logged-in customer.
GET	/tickets/queue	Agent	Retrieves paginated queue of agent's assigned tickets.
GET	/tickets/{id}	Owner / Agent	Fetches full ticket details and thread comments.
POST	/tickets/{id}/respond	Owner / Agent	Posts reply message into ticket conversation thread.
PATCH	/tickets/{id}/status	Agent / Admin	Updates state (ASSIGNED → IN_PROGRESS → RESOLVED).
PATCH	/tickets/{id}/assign	Admin	Assigns or reassigns ticket to target agent ID.
PATCH	/tickets/{id}/reopen	Customer	Reopens ticket with mandatory reason comment.
PATCH	/tickets/{id}/close	Customer	Customer accepts fix and transitions ticket to CLOSED.

5. Relational Data Dictionary

USERS Table

Column Name	Data Type	Key / Constraints
id	BIGINT (int8)	PRIMARY KEY GENERATED AS IDENTITY
name	VARCHAR(100)	NOT NULL
email	VARCHAR(100)	UNIQUE, NOT NULL
password_hash	VARCHAR(255)	NOT NULL
role	VARCHAR(20)	CHECK (role IN ('customer', 'agent', 'admin'))
created_at	TIMESTAMPTZ	DEFAULT NOW()

TICKETS Table

Column Name	Data Type	Key / Constraints
id	BIGINT (int8)	PRIMARY KEY GENERATED AS IDENTITY
customer_id	BIGINT	NOT NULL, FK -> users(id) ON DELETE CASCADE
title	VARCHAR(255)	NOT NULL
category	VARCHAR(50)	NOT NULL
priority	VARCHAR(20)	DEFAULT 'medium'
description	TEXT	NOT NULL
status	VARCHAR(30)	DEFAULT 'open' CHECK (status IN (...))
assigned_agent_id	BIGINT	FK -> users(id) ON DELETE SET NULL
created_at	TIMESTAMPTZ	DEFAULT NOW()

RESPONSES Table

Column Name	Data Type	Key / Constraints
id	BIGINT (int8)	PRIMARY KEY GENERATED AS IDENTITY
ticket_id	BIGINT	NOT NULL, FK -> tickets(id) ON DELETE CASCADE
sender_id	BIGINT	NOT NULL, FK -> users(id) ON DELETE CASCADE
message	TEXT	NOT NULL
created_at	TIMESTAMPTZ	DEFAULT NOW()

TICKET_STATUS_HISTORY Table

Column Name	Data Type	Key / Constraints
id	BIGINT (int8)	PRIMARY KEY GENERATED AS IDENTITY
ticket_id	BIGINT	NOT NULL, FK -> tickets(id) ON DELETE CASCADE
old_status	VARCHAR(30)	NULLABLE
new_status	VARCHAR(30)	NOT NULL
changed_by	BIGINT	NOT NULL, FK -> users(id)
created_at	TIMESTAMPTZ	DEFAULT NOW()